

DOCUMENT 2: COMPLETE GUIDE TO EVERY FILE IN THIS PROJECT

Simple English Guide: HOW, WHY, and WHEN for Every Project File

Introduction

This document breaks down every single file in the **Loan Amount Prediction Machine Learning System**. Each file is described in plain English with:

- **WHAT is this file?** Its purpose and role in the system.
 - **HOW does it work?** Step-by-step code and workflow explanation.
 - **WHY was it created?** Why this file is necessary for the project.
 - **WHEN should you use or modify it?** Real-world developer guidelines.
-

File 1: `app.py` (Streamlit Web Dashboard)

1. WHAT is this file?

`app.py` is the front-facing user interface and web application. It connects human users (loan applicants and bank credit officers) to our trained machine learning models using **Streamlit**.

2. HOW does it work?

- 1. Configures the Dashboard:** Sets the title, icon, and custom CSS styling for a dark-mode fintech appearance.
- 2. Builds the Navigation Sidebar:** Creates 5 tabs: Loan Predictor, 6-Model Benchmark, Exploratory Data Analysis, Executive Q&A, and PDF Downloads.
- 3. Renders the Input Form:** Collects applicant details (income, credit score slider, requested loan amount, liabilities, collateral value, employment type, location).
- 4. Provides 1-Click Demo Profiles:** Allows users to auto-fill profiles like "Tech Salaried", "Business Owner", or "Rural Borrower".
- 5. Calls the Inference Engine:** Calls `predict_loan_amount()` from `src/predict.py`.
- 6. Displays Results:** Formats the output prominently as **Predicted Loan Amount: ■X** (in Lakhs/Crores), displays a color-coded credit risk badge, calculates FOIR and maximum monthly EMI capacity, and renders an on-the-fly comparison table showing what all 6 algorithms predict for that same applicant!

3. WHY was it created?

Machine learning code stored in a terminal or script cannot be used by banking staff. `app.py` turns code into an intuitive, real-time software product.

4. WHEN should you run or modify it?

- **Run:** Anytime you want to launch and test the user interface (`streamlit run app.py`).
 - **Modify:** When you want to add new input fields, change styling, add extra tabs, or customize underwriter advisory rules.
-

File 2: `data/real_loan_dataset.csv` (Raw Online Dataset)

1. WHAT is this file?

This is the authentic, real-world online dataset sourced directly from the **HackerEarth / Kaggle "Predict the Loan Sanction Amount"** challenge. It contains **30,000 raw application records** with 24 columns.

2. HOW does it work?

It contains raw columns including: `Customer ID`, `Name`, `Gender`, `Age`, `Income (USD)`, `Income Stability`, `Profession`, `Type of Employment`, `Location`, `Loan Amount Request (USD)`, `Current Loan Expenses (USD)`, `Dependents`, `Credit Score`, `No. of Defaults`, `Has Active Credit Card`, `Property Price`, `Co-Applicant`, and `Loan Sanction Amount (USD)`.

3. WHY was it created?

To fulfill the requirement: *"do not make your dataset, find online dataset for project"*. This ensures all statistical correlations, credit scores, and sanction decisions reflect genuine real-world banking applications.

4. WHEN should you use it?

It serves as the immutable ground truth raw dataset. It should not be edited manually; data cleaning should always be performed programmatically via code.

File 3: `data/process_online_dataset.py` (Data Pipeline Script)

1. WHAT is this file?

A Python cleaning and data preparation script that takes `data/real_loan_dataset.csv` and transforms it into the clean, production-ready dataset `data/loan_data.csv`.

2. HOW does it work?

- 1. Filters Invalid Records:** Removes rows where the sanctioned loan amount is zero or negative (rejected loans or system errors), keeping genuine positive loan sanctions.
- 2. Handles Missing Values:** Uses median imputation for numerical columns (**Income**, **Credit Score**, **Current Expenses**, **Dependents**) and categorical defaults ('Other', 'Unpossessed').
- 3. Converts Currency to Indian Rupees (₹):** Multiplies USD amounts by ₹80 to match standard Indian banking parameters requested in the prompt (**Predicted Loan Amount: ₹X**).
- 4. Samples 10,000 Records:** Selects a clean, statistically representative 10,000-row dataset for high-speed, robust training and validation.
- 5. Exports Clean CSV:** Saves the processed data to **data/loan_data.csv**.

3. WHY was it created?

Raw data cannot be passed directly into scikit-learn models without causing crashes due to null values, string errors, or negative targets. This script automates reproducible data preparation.

4. WHEN should you run or modify it?

- **Run:** When you download a new batch of raw data and need to refresh the training dataset (**python3 data/process_online_dataset.py**).
 - **Modify:** If you want to change imputation rules, adjust currency conversion, or include additional raw columns.
-

File 4: data/loan_data.csv (Clean Production Dataset)

1. WHAT is this file?

The cleaned, validated tabular dataset used to train and test our 6 machine learning models. It has **10,000 rows** and **15 feature columns** with zero missing values.

2. HOW does it work?

Contains 14 predictor features (**ApplicantIncome**, **IncomeStability**, **Profession**, **EmploymentType**, **Location**, **LoanAmountRequest**, **ExistingLiabilities**, **Dependents**, **CreditScore**, **DefaultsCount**, **ActiveCreditCard**, **PropertyLocation**, **CoApplicant**, **AssetValue**) and 1 target variable (**LoanAmount**).

3. WHY was it created?

To provide a clean, standardized, high-quality data foundation for exploratory data analysis and model training.

4. WHEN should you use it?

Both **src/eda.py** and **src/train_models.py** read this file as their primary input.

File 5: `src/eda.py` (Exploratory Data Analysis Script)

1. WHAT is this file?

A Python data visualization and statistical analysis script. It analyzes the relationships in `loan_data.csv` and outputs publication-quality visualization charts into the `plots/` folder.

2. HOW does it work?

1. Loads `data/loan_data.csv` using Pandas.
2. Computes summary statistics, null value counts, and Pearson correlation coefficients.
3. Generates and saves 6 high-resolution charts:
 - `loan_amount_distribution.png`: Raw vs Log-transformed target distributions.
 - `correlation_heatmap.png`: Full correlation matrix heatmap of numeric features.
 - `request_vs_sanctioned.png`: Regression scatter plot showing requested vs sanctioned amounts.
 - `credit_score_impact.png`: Boxplots of loan amounts across credit tiers.
 - `income_asset_impact.png`: Dual scatter plots of income and collateral property values.
 - `location_defaults_impact.png`: Bar plots showing the effect of defaults and geographic locations.

3. WHY was it created?

Before training any model, data scientists must visually inspect the data to discover non-linearities, detect outliers, and understand which features have the strongest predictive power.

4. WHEN should you run or modify it?

- **Run:** `python3 src/eda.py` whenever you update the dataset or want to regenerate charts.
- **Modify:** If you want to add new plots (e.g., age distributions or pairplots).

File 6: `src/train_models.py` (Model Training & Comparative Benchmark)

1. WHAT is this file?

The core machine learning engine of the project. It builds pipelines, trains all 6 regression algorithms, compares their performance, selects the best model, and saves the trained artifacts.

2. HOW does it work?

1. **Splits Data:** 80% Training ($N=8,000$), 20% Testing ($N=2,000$).

2. Defines Preprocessing: A `ColumnTransformer` that standardizes numerical features (`StandardScaler`) and one-hot encodes categories (`OneHotEncoder`).

3. Builds 4 Model Pipelines:

- Linear Regression
- Decision Tree Regression
- Random Forest Regression
- Gradient Boosting Regression

4. Calculates Evaluation Metrics: Computes MAE, MSE, RMSE, and R^2 on both training and test sets.

5. Ranks Models: Identifies the champion model with the lowest RMSE.

6. Saves Artifacts: Saves models as `.joblib` files, writes benchmark JSON files, and generates evaluation charts (`actual_vs_predicted.png`, `residuals_distribution.png`, `feature_importance.png`, `model_comparison_metrics.png`).

3. WHY was it created?

It executes the core mathematical and experimental requirements of the project: developing 6 regression models and conducting a rigorous comparative benchmark.

4. WHEN should you run or modify it?

- **Run:** `python3 src/train_models.py` to train/retrain the models.
- **Modify:** To tune hyperparameters (e.g., change tree depth, learning rates, or polynomial degree).

File 7: `src/predict.py` (Inference & Underwriting Engine)

1. WHAT is this file?

A lightweight Python module designed to take a dictionary of applicant inputs, run inference through our saved machine learning models, and compute banking credit metrics.

2. HOW does it work?

- 1. Loads Models on Demand:** Loads `.joblib` files from the `models/` directory and caches them in memory.
- 2. Formats Currency:** Implements `format_inr()` to convert raw numbers into standard Indian numbering format (e.g., ₹50,11,855) and `format_inr_words()` (e.g., "50.12 Lakh").
- 3. Calculates Underwriting Ratios:** Computes Fixed Obligation to Income Ratio (FOIR %) and Maximum Disposable EMI Capacity.
- 4. Multi-Model Inference:** Generates predictions from all 6 models simultaneously for comparative analysis.

3. WHY was it created?

To separate model inference from the user interface. This allows `predict.py` to be imported by the Streamlit app, a REST API, or an automated batch script without code duplication.

4. WHEN should you run or modify it?

- **Modify:** If you want to change financial ratio thresholds or add new underwriting sanity checks.
-

File 8: `models/best_model.joblib` (Champion Model Pipeline)

1. WHAT is this file?

A serialized binary file containing the complete, pre-trained **Gradient Boosting Regression** pipeline (including fitted scaler, encoder, and decision trees).

2. HOW does it work?

Python's `joblib` library serializes the fitted mathematical weights into a compact file on disk. When loaded, it can instantly predict loan amounts in milliseconds without needing to retrain from scratch.

3. WHY was it created?

Training models takes time and computation. Serializing the model allows instant deployment in production.

4. WHEN should you use it?

Loaded automatically whenever the application makes predictions.

File 9: `models/metrics_comparison.json` & `models/model_metadata.json`

1. WHAT are these files?

- `metrics_comparison.json`: A structured JSON file storing the exact MAE, MSE, RMSE, and R^2 scores for all 6 models.
- `model_metadata.json`: Stores feature names, categorical choices, and training record counts.

2. HOW do they work?

They store machine-readable metadata that external dashboards and monitoring services can consume without loading large binary files.

3. WHY were they created?

Provides clean separation between machine learning evaluation results and application logic.

File 10: `plots/*.png` (Visual Charts Directory)

1. WHAT is this directory?

A dedicated folder containing all 10 high-resolution charts generated during EDA and model evaluation:

- `correlation_heatmap.png`
- `loan_amount_distribution.png`
- `request_vs_sanctioned.png`
- `credit_score_impact.png`
- `income_asset_impact.png`
- `location_defaults_impact.png`
- `model_comparison_metrics.png`
- `actual_vs_predicted.png`
- `residuals_distribution.png`
- `feature_importance.png`

2. HOW does it work?

Streamlit loads and displays these charts directly in the "■ 6-Model Benchmark" and "■ Exploratory Data Analysis" tabs.

3. WHY was it created?

Visual charts are crucial for executive reporting, academic presentation, and stakeholder comprehension.

File 11: `final_analysis_report.md` (Executive Analysis Report)

1. WHAT is this file?

A comprehensive technical research document that covers the problem statement, algorithm mathematics, comparative performance, Section 7 analysis, and detailed answers to all 6 core questions from Section 8.

2. HOW does it work?

Written in GitHub-flavored markdown with mathematical formulas, ASCII architecture diagrams, benchmark tables, and policy recommendations.

3. WHY was it created?

To fulfill the project requirements for written documentation, model analysis, and answering the institution's key business questions.

File 12: README .md (Project Manual)

1. WHAT is this file?

The primary landing page and documentation manual for the repository.

2. HOW does it work?

Explains project architecture, setup instructions, dependencies, and command-line execution steps for any developer cloning the repository.